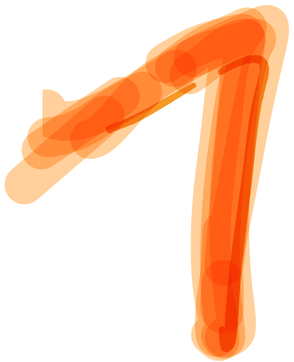
西南交通大学
Southwest Jiaotong University

Concurrency: Deadlock and Starvation



Xiangyi Chen

xiangyichen@swjtu.edu.cn

Outline

- Principles of Deadlock
- Deadlock Prevention
- Deadlock Avoidance
- Deadlock Detection
- Dining Philosophers Problem
- Summary

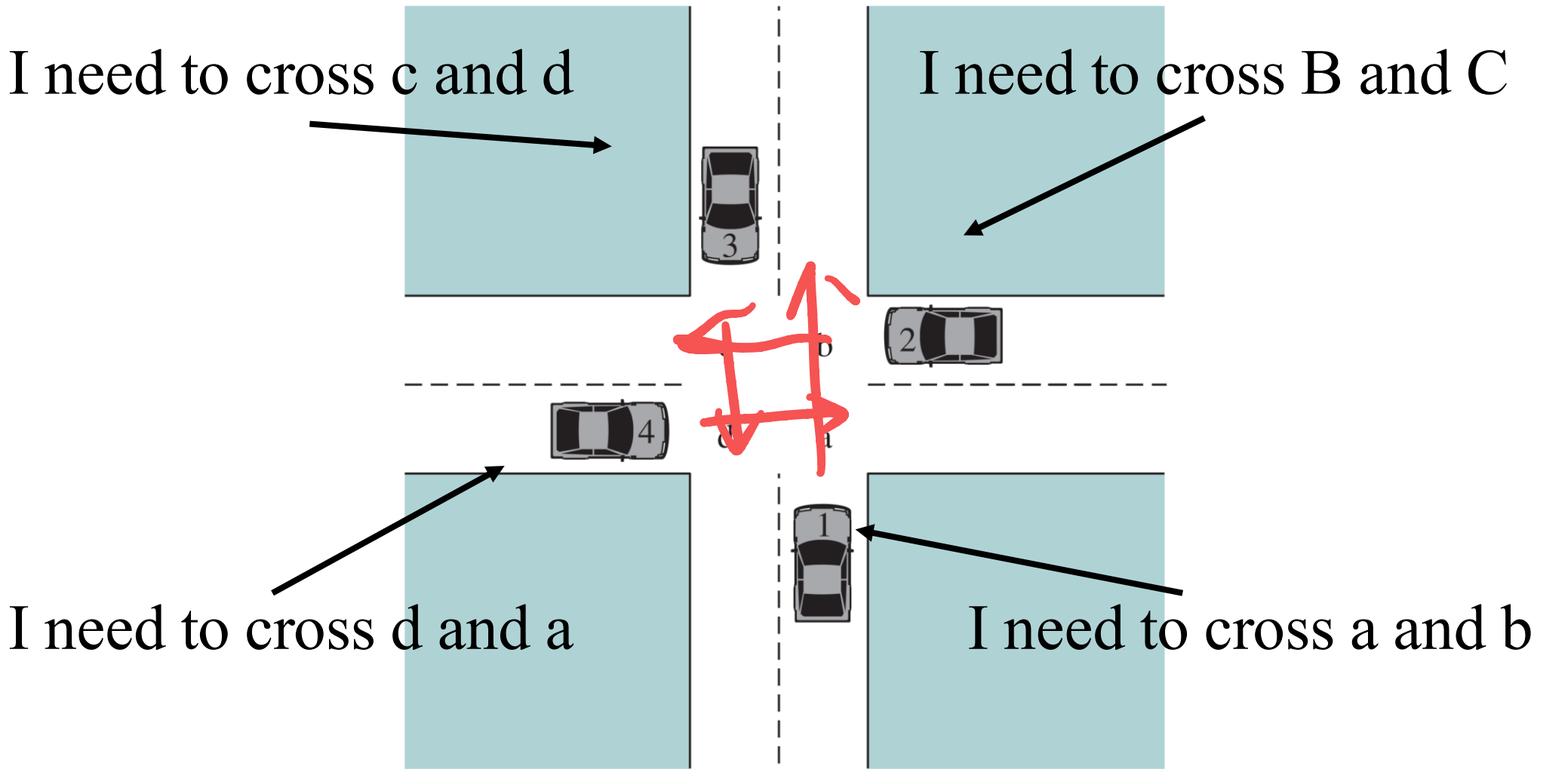
6.1 Deadlock

- The permanent blocking of a set of processes since competing for system resources or communicate with each other
- Each process in the set is blocked awaiting an event that can only be triggered by another blocked process in the set
- No efficient solution

Potential Deadlock

- The rule of the road in the United States is that a car at a four-way stop should defer to a car immediately to its right.

让行其右侧的车辆

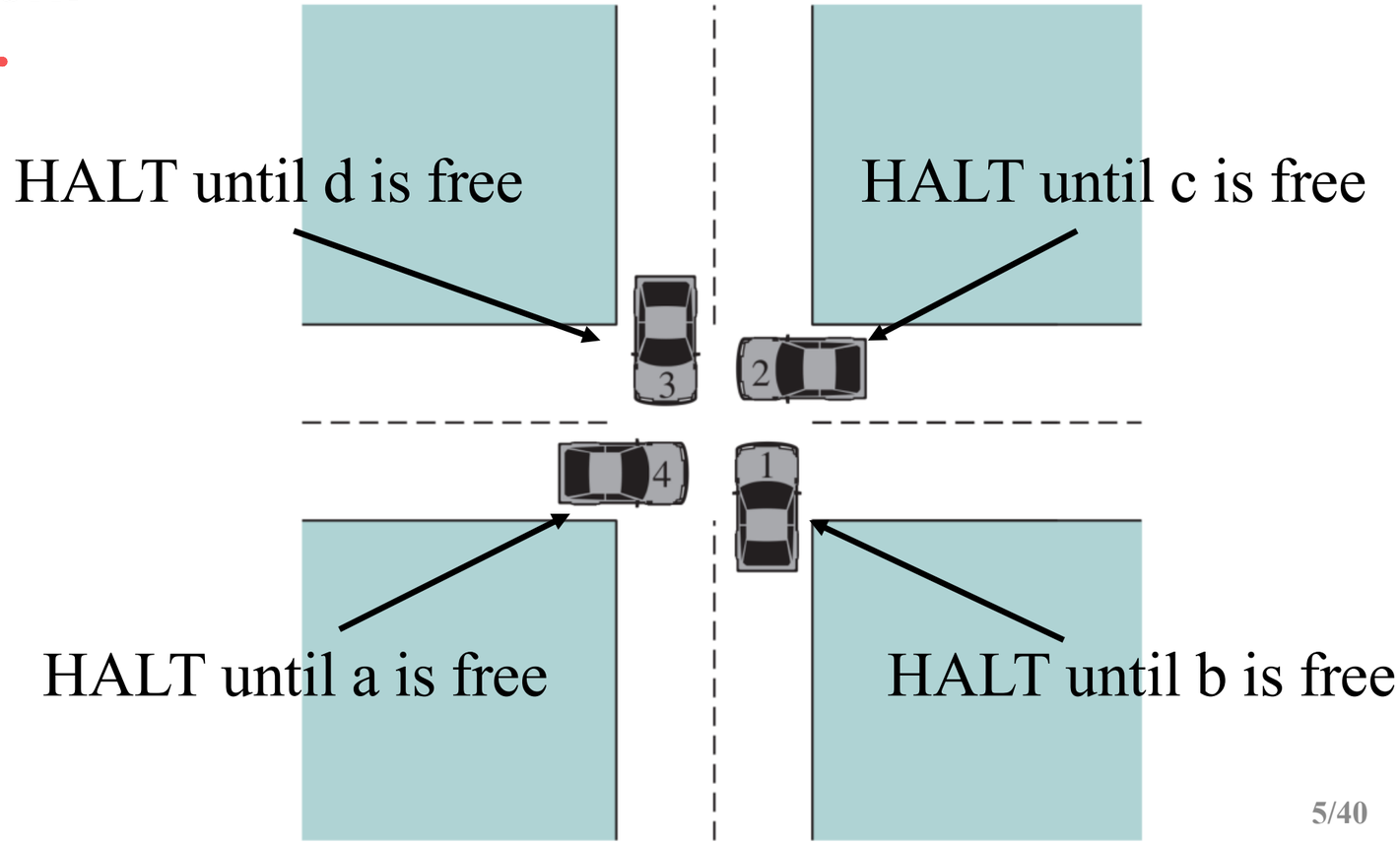


Actual Deadlock

hold and wait

占用一个资源

- **Each car seizes one resource (one quadrant) but cannot proceed because the required second resource has already been seized by another car.**
- **This is an actual deadlock.**

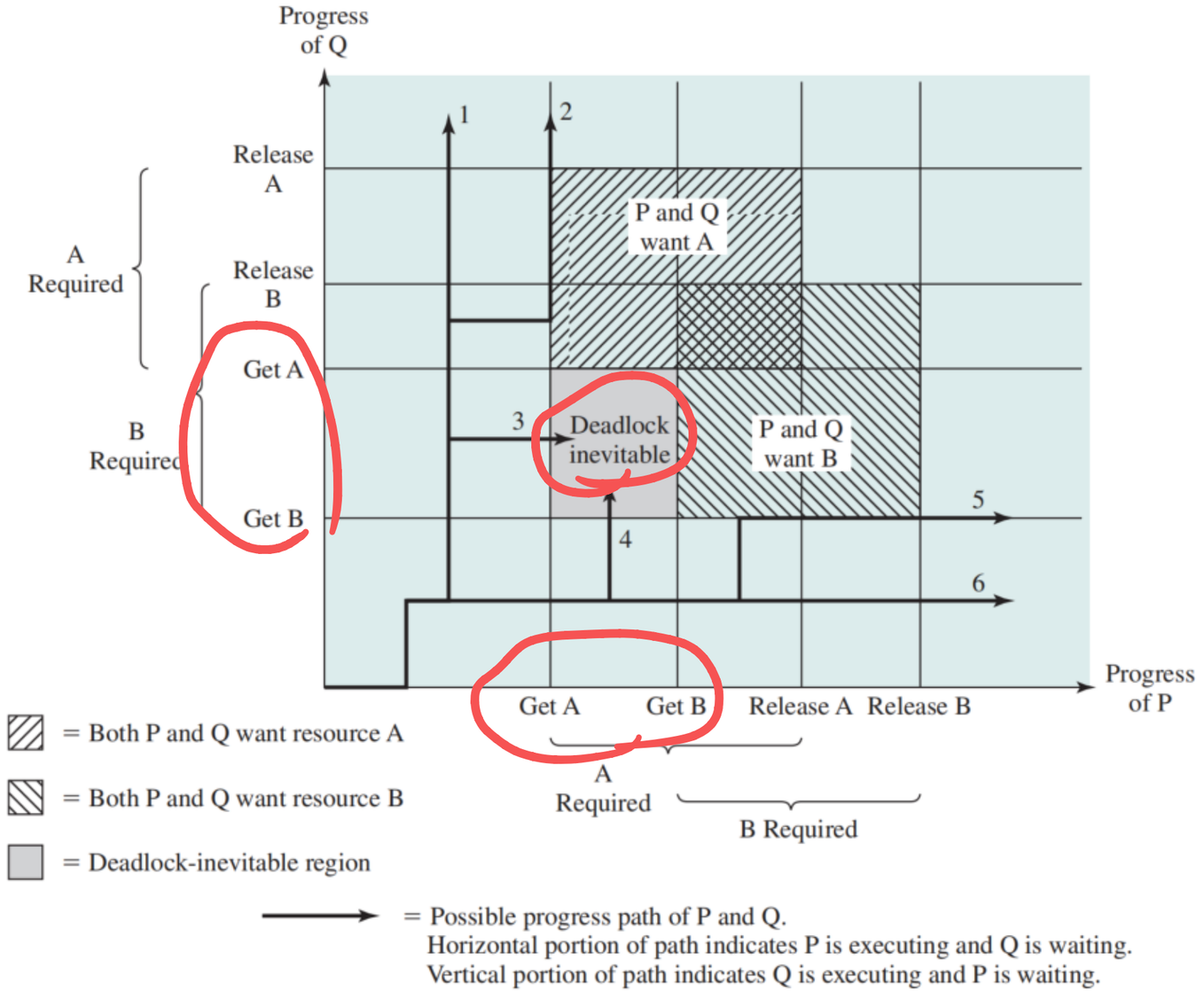


Joint Progress Diagram

可以通过速度优先级避免死锁

- A depiction of deadlock involving processes and computer resources
- illustrates the progress of two processes competing for two resources

Process P	Process Q
• • •	• • •
Get A	Get B
• • •	• • •
Get B	Get A
• • •	• • •
Release A	Release B
• • •	• • •
Release B	Release A
• • •	• • •



No Deadlock Example

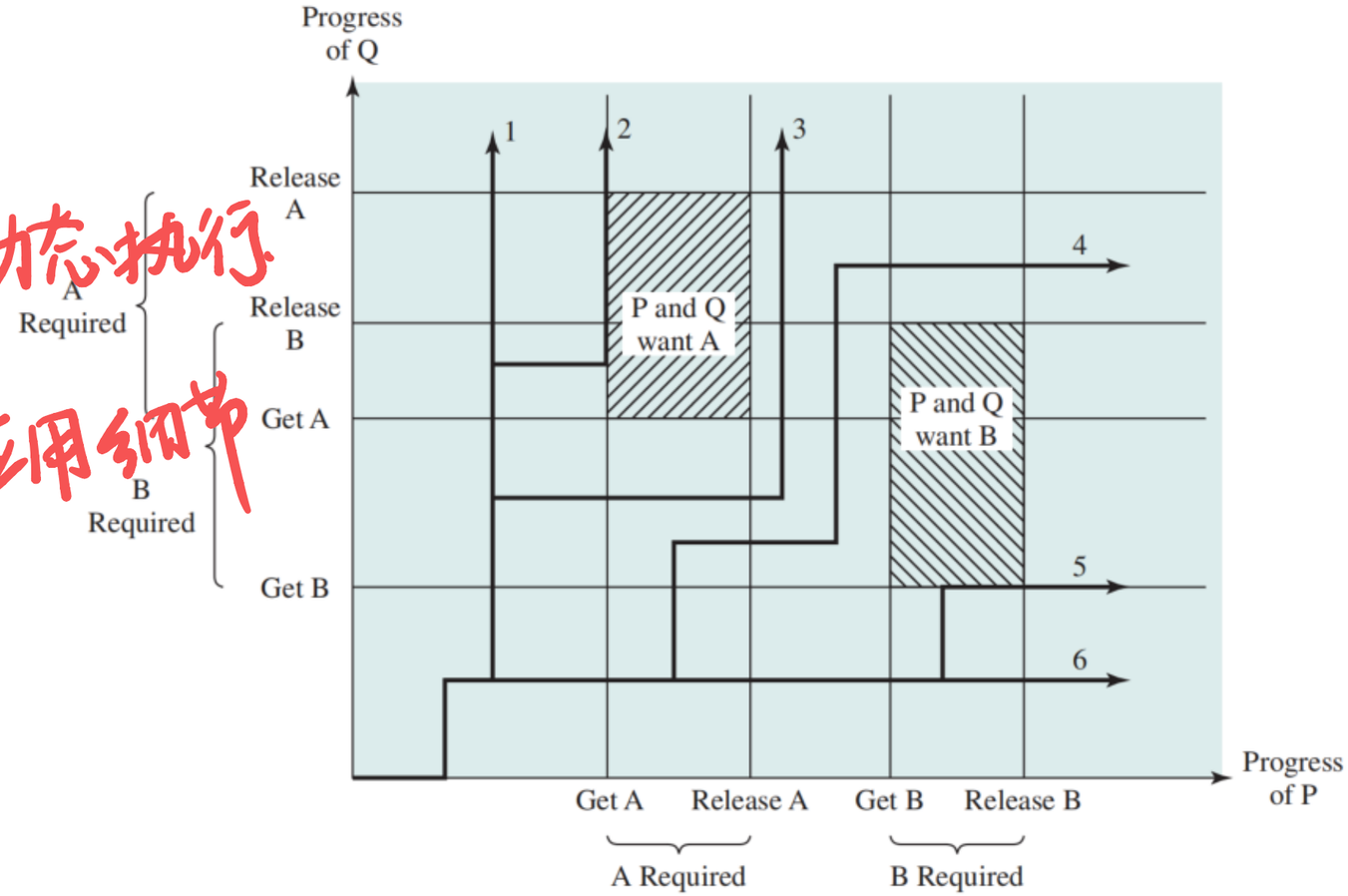
无死锁

- Whether or not deadlock occurs depends on both the dynamics of the execution and on the details of the application.

动态执行

应用细节

Process P	Process Q
• • •	• • •
Get A	Get B
• • •	• • •
Release A	Get A
• • •	• • •
Get B	Release B
• • •	• • •
Release B	Release A
• • •	• • •



Resource Categories

Reusable

可重用资源

- can be safely used by only one process at a time and is not deleted by that use
- processors, I/O channels, main and secondary memory, devices, files

…主存…

Consumable

消耗性资源

- one that can be created (produced) and destroyed (consumed)
- interrupts, signals, messages

会被删除掉.

Example 1: reusable resource

- a disk file D and a tape drive T

Step	Process P Action
p_0	Request (D)
p_1	Lock (D)
p_2	Request (T)
p_3	Lock (T)
p_4	Perform function
p_5	Unlock (D)
p_6	Unlock (T)

Step	Process Q Action
q_0	Request (T)
q_1	Lock (T)
q_2	Request (D)
q_3	Lock (D)
q_4	Perform function
q_5	Unlock (T)
q_6	Unlock (D)

Two Processes Competing for Reusable Resources

While execute p0 p1 q0 q1 p2 q2, deadlock or not?

Example 2: Memory Request

- Space is available for allocation of 200Kbytes, and the following sequence of events occur:

P1	P2
. . .	. . .
Request 80 Kbytes;	Request 70 Kbytes;
. . .	. . .
Request 60 Kbytes;	Request 80 Kbytes;

- Deadlock or not? R1(p1), R1(p2), R2(p2), R2(p1)?

Consumable Resources Deadlock

- Consider a pair of processes, in which each process attempts to receive a message from the other process and then send a message to the other process:

P1	P2
. . .	. . .
Receive (P2);	Receive (P1);
. . .	. . .
Send (P2, M1);	Send (P1, M2);

- Deadlock occurs if the Receive is blocking

Resource Allocation Graphs 资源分配图

西南交通大学 Southwest Jiaotong University

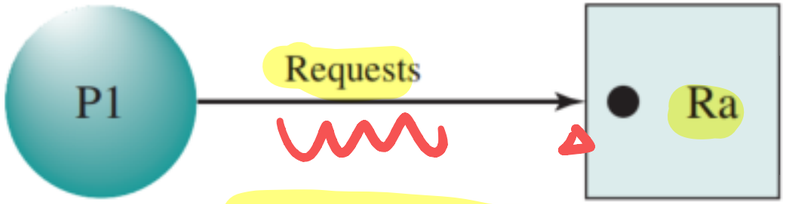
- A useful tool in characterizing the allocation of resources to processes is the resource allocation graph

描述系统资源分配的有效工具

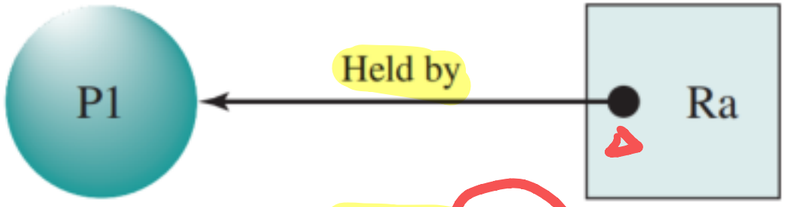
- The resource allocation graph is a directed graph that depicts a state of the system of resources and processes, with each process and each resource represented by a node

资源和进程均用节点表示

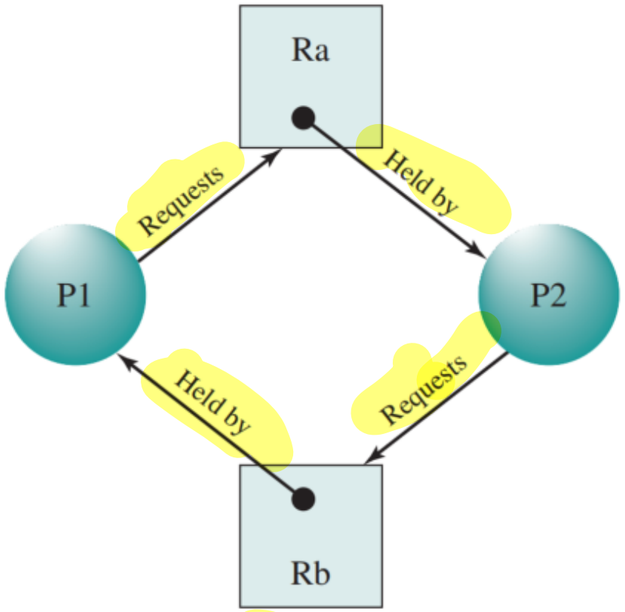
Resource Allocation Graphs



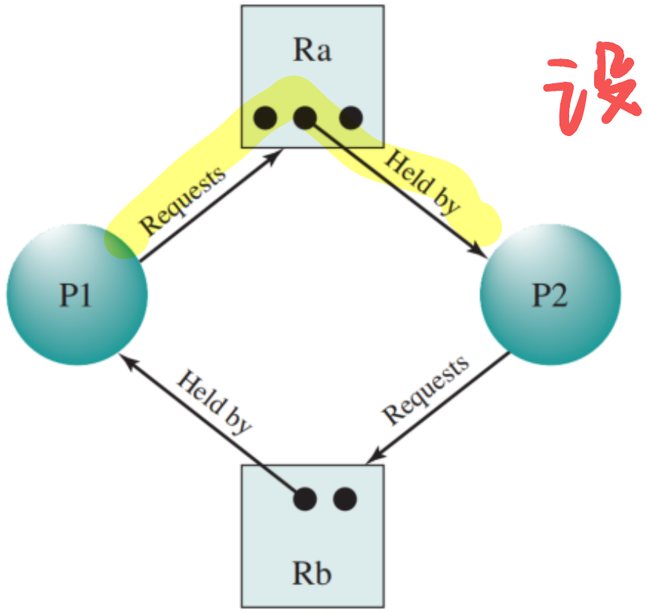
(a) Resource is requested



(b) Resource is held



(c) Circular wait

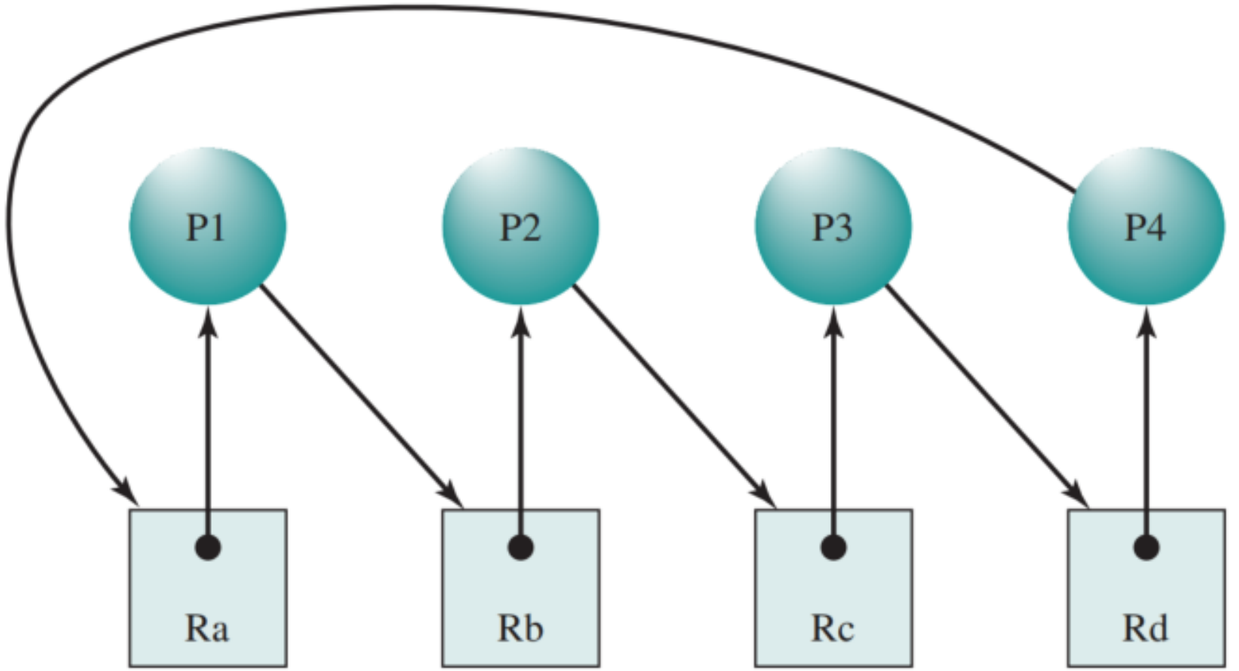


(d) No deadlock

设多个占用点

Resource Allocation Graphs

- **Circular wait**. A closed chain of processes exists, such that **each process holds at least one resource needed by the next process** in the chain



Resource Allocation Graph for Figure 6.1b

Conditions for Deadlock

Necessary Conditions for a deadlock:

- **Mutual Exclusion**
  - only one process may use a resource at a time
- **Hold and Wait**
  - a process may hold allocated resources while awaiting assignment of others
- **No Preemption**
  - no resource can be forcibly removed from a process holding it

Sufficient Condition for a deadlock:

- **Circular Wait**
  - a closed chain of processes exists, such that each process holds at least one resource needed by the next process in the chain

Summary

Possibility of Deadlock	Existence of Deadlock
1. Mutual exclusion	1. Mutual exclusion
2. No preemption	2. No preemption
3. Hold and wait	3. Hold and wait
	4. Circular wait

6.2 Three Approaches Dealing with Deadlock

西南交通大学 Southwest Jiaotong University

- **Deadlock prevention**: Disallow one of the three necessary conditions for deadlock occurrence, or prevent circular wait condition from happening.
- **Deadlock avoidance**: Do not grant a resource request if this allocation might lead to deadlock.
- **Deadlock detection**: Grant resource requests when possible, but periodically check for the presence of deadlock and take action to recover.

Deadlock Prevention

西南交通大学 Southwest Jiaotong University

Mutual Exclusion

- In general, the first of the four listed conditions cannot be disallowed.

Hold and Wait

- require that a process request all of its required resources at one time and blocking the process until all requests can be granted simultaneously
- Inefficient: a long time waiting and some resource may be unused during the allocation

No Preemption

- if a process holding certain resources is denied a further request, that process must release its original resources and request them again
- if a process requests a resource that is currently held by another process, OS may preempt the second process and require it to release its resources

Circular Wait

- define a linear ordering of resource types. If a process has been allocated resources of type R, then it may subsequently request only those resources of types following R in the ordering.
- resource R_i precedes R_j in the ordering if i < j.

Deadlock Avoidance

西南交通大学 Southwest Jiaotong University

- Deadlock prevent leads to inefficient use of resource and inefficient execution of processes.

- Deadlock avoidance: allows the three necessary conditions but makes judicious choices to assure that the deadlock point is never reached.

谨慎选择

- As such, avoidance allows more concurrency than prevention.

- Requires knowledge of future process requests

Two Approaches to Deadlock Avoidance

Process Initiation Denial 拒绝

- do not start a process if its demands might lead to deadlock

Resource Allocation Denial

- do not grant an incremental resource request to a process if this allocation might lead to deadlock
- Referred to as **the banker algorithm**
- State of the system reflects the current allocation of resources to processes
- Safe state is one in which there is at least one sequence of resource allocations to processes that does not result in a deadlock
- Unsafe state is a state that is not safe

Process Initiation Denial

- **The following relationships hold**

$R_j = V_j + \sum_{i=1}^{n} A_{ij}, \quad \text{fc}$ All resources are either available or allocated.

$C_{ij} \leq R_j, \quad \text{for all } i, j$ No process can claim more than the total amount of resources in the system.

$A_{ij} \leq C_{ij}, \quad \text{for all } i, j$ No process is allocated more resources of any type than the process originally claimed to need.

Resource $= \mathbf{R} = (R_1, R_2, \ldots, R_m)$	Total amount of each resource in the system
Available $= \mathbf{V} = (V_1, V_2, \ldots, V_m)$	Total amount of each resource not allocated to any process
Claim $= \mathbf{C} = \begin{pmatrix} C_{11} & C_{12} & \ldots & C_{1m} \\ C_{21} & C_{22} & \ldots & C_{2m} \\ \vdots & \vdots & \vdots & \vdots \\ C_{n1} & C_{n2} & \ldots & C_{nm} \end{pmatrix}$	C_{ij} = requirement of process i for resource j
Allocation $= \mathbf{A} = \begin{pmatrix} A_{11} & A_{12} & \ldots & A_{1m} \\ A_{21} & A_{22} & \ldots & A_{2m} \\ \vdots & \vdots & \vdots & \vdots \\ A_{n1} & A_{n2} & \ldots & A_{nm} \end{pmatrix}$	A_{ij} = current allocation to process i of resource j

Start a new process

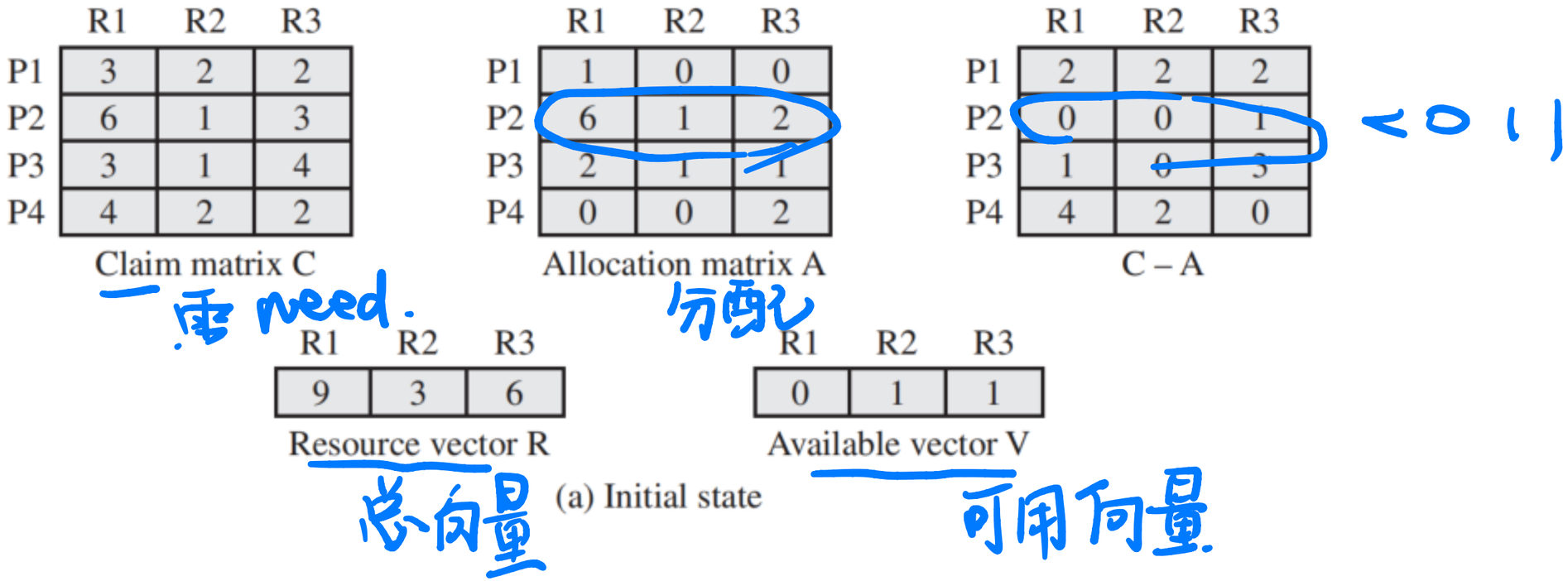
$$R_j \geq C_{(n+1)j} + \sum_{i=1}^{n} C_{ij}, \quad \text{for all } j$$

Resource Allocation Denial

- do not grant an incremental resource request to a process if this allocation might lead to deadlock
- State of the system reflects the current allocation of resources to processes
- Safe state is one in which there is at least one sequence of resource allocations to processes that does not result in a deadlock
- Unsafe state is a state that is not safe

Determination of a Safe State

- State of a system consisting of four processes and three resources
- Allocations have been made to the four processes



	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Claim matrix C

	R1	R2	R3
P1	1	0	0
P2	6	1	2
P3	2	1	1
P4	0	0	2

Allocation matrix A

	R1	R2	R3
P1	2	2	2
P2	0	0	1
P3	1	0	3
P4	4	2	0

C – A

R1	R2	R3
9	3	6

Resource vector R

R1	R2	R3
0	1	1

Available vector V

(a) Initial state

P2 Runs to Completion

	R1	R2	R3
P1	3	2	2
P2	0	0	0
P3	3	1	4
P4	4	2	2

Claim matrix C

	R1	R2	R3
P1	1	0	0
P2	0	0	0
P3	2	1	1
P4	0	0	2

Allocation matrix A

	R1	R2	R3
P1	2	2	2
P2	0	0	0
P3	1	0	3
P4	4	2	0

C – A

R1	R2	R3
9	3	6

Resource vector R

R1	R2	R3
6	2	3

Available vector V

(b) P2 runs to completion

Allocation + Work

P1 Runs to Completion

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	3	1	4
P4	4	2	2

Claim matrix C

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	2	1	1
P4	0	0	2

Allocation matrix A

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	1	0	3
P4	4	2	0

C – A

R1	R2	R3
9	3	6

Resource vector R

R1	R2	R3
7	2	3

Available vector V

(c) P1 runs to completion

P3 Runs to Completion

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	0	0	0
P4	4	2	2

Claim matrix C

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	0	0	0
P4	0	0	2

Allocation matrix A

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	0	0	0
P4	4	2	0

C – A

R1	R2	R3
9	3	6

Resource vector R

R1	R2	R3
9	3	4

Available vector V

(d) P3 runs to completion

Thus, the state defined originally is a safe state.
The sequence is P2, P1,P3,P4

Determination of an Unsafe State

	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Claim matrix C

	R1	R2	R3
P1	1	0	0
P2	5	1	1
P3	2	1	1
P4	0	0	2

Allocation matrix A

	R1	R2	R3
P1	2	2	2
P2	1	0	2
P3	1	0	3
P4	4	2	0

C – A

R1	R2	R3
9	3	6

Resource vector R

R1	R2	R3
1	1	2

Available vector V

(a) Initial state

	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Claim matrix C

	R1	R2	R3
P1	2	0	1
P2	5	1	1
P3	2	1	1
P4	0	0	2

Allocation matrix A

	R1	R2	R3
P1	1	2	1
P2	1	0	2
P3	1	0	3
P4	4	2	0

C – A

R1	R2	R3
9	3	6

Resource vector R

R1	R2	R3
0	1	1

Available vector V

(b) P1 requests one unit each of R1 and R3

Quiz

6.1 Consider the following snapshot of a system. There are no outstanding unsatisfied requests for resources.

available

r1	r2	r3	r4
1	5	2	0

	current allocation				maximum demand			
process	r1	r2	r3	r4	r1	r2	r3	r4
p0	0	0	1	2	0	0	1	2
p1	1	0	0	1	1	5	1	1
p2	1	3	0	4	2	3	5	6
p3	0	6	3	2	0	6	7	2
p4	0	0	1	4	0	6	5	6

Is the system safe?

Deadlock Avoidance Logic

```
struct  state {
            int  resource[m];
            int  available[m];
            int  claim[n][m];
            int  alloc[n][m];
}
```

(a) Global data structures

```
if (alloc [i,*] + request [*] > claim [i,*])
      <error>;                          /* total request > claim*/
else if (request [*] > available [*])
     <suspend process>;
else  {                                 /* simulate alloc */
     <define newstate by:
     alloc [i,*] = alloc [i,*] + request [*];
     available [*] = available [*] - request [*]>;
}
if  (safe (newstate))
       <carry out allocation>;
else  {
       <restore original state>;
       <suspend process>;
}
```

(b) Resource allocation algorithm

Deadlock Avoidance Logic（银行家算法）


```
boolean safe (state S) {
    int currentavail[m];
    process rest[<number of processes>];
    currentavail = available;
    rest = {all processes};
    possible = true;
    while (possible) {
        <find a process Pk in rest such that
        claim [k,*] - alloc [k,*]<= currentavail;
        if (found) {          /* simulate execution of Pk */
            currentavail = currentavail + alloc [k,*];
            rest = rest - {Pk};
        }
        else  possible = false;
    }
    return (rest == null);
}
```

(c) Test for safety algorithm (banker's algorithm)

Deadlock Avoidance Advantages

西南交通大学 Southwest Jiaotong University

Advantages

- It is not necessary to preempt and rollback processes, as in deadlock prevention
- It is less restrictive than deadlock prevention

Restrictions

- Maximum resource requirement for each process must be stated in advance
- Processes under consideration must be independent and with no synchronization requirements
- There must be a fixed number of resources to allocate
- No process may exit while holding resources

Deadlock Detection

- **Deadlock prevention strategies are very conservative （保守）**
  - limit access to resources by imposing restrictions on processes
- **Deadlock detection strategies do the opposite**
  - resource requests are granted whenever possible
- **A check for deadlock can be made as frequently as each resource request or, less frequently, depending on how likely it is for a deadlock to occur.**

Deadlock Detection Strategy

请求矩阵Q

- **A request matrix Q is defined such that Q_{ij} represents the amount of resources of type j requested by process i.**

1. Mark each process that has a row in the Allocation matrix of all zeros. A process that has no allocated resources cannot participate in a deadlock.
2. Initialize a temporary vector W to equal the Available vector.
3. Find an index i such that process i is currently unmarked and the i-th row of Q is less than or equal to W. That is, $Q_{ik} \leq W_k$, for $1 \leq k \leq m$. If no such row is found, terminate the algorithm.
4. If such a row is found, mark process i and add the corresponding row of the allocation matrix to W. That is, set $W_k = W_k + A_{ik}$, for 1 ... k ... m. Return to step 3.

A deadlock exists if and only if there are unmarked processes at the end of the algorithm

Examples of Deadlock Detection

- P4, P3 are marked, then terminated.
- P1, P2 are unmarked.

	R1	R2	R3	R4	R5
P1	0	1	0	0	1
P2	0	0	1	0	1
P3	0	0	0	0	1
P4	1	0	1	0	1

Request matrix Q

need

	R1	R2	R3	R4	R5
P1	1	0	1	1	0
P2	1	1	0	0	0
P3	0	0	0	1	0
P4	0	0	0	0	0

Allocation matrix A

2 1 1 2 0

R1	R2	R3	R4	R5
2	1	1	2	1

Resource vector

R1	R2	R3	R4	R5
0	0	0	0	1

Available vector

P4.

6.3 Recovery Strategies

西南交通大学 Southwest Jiaotong University

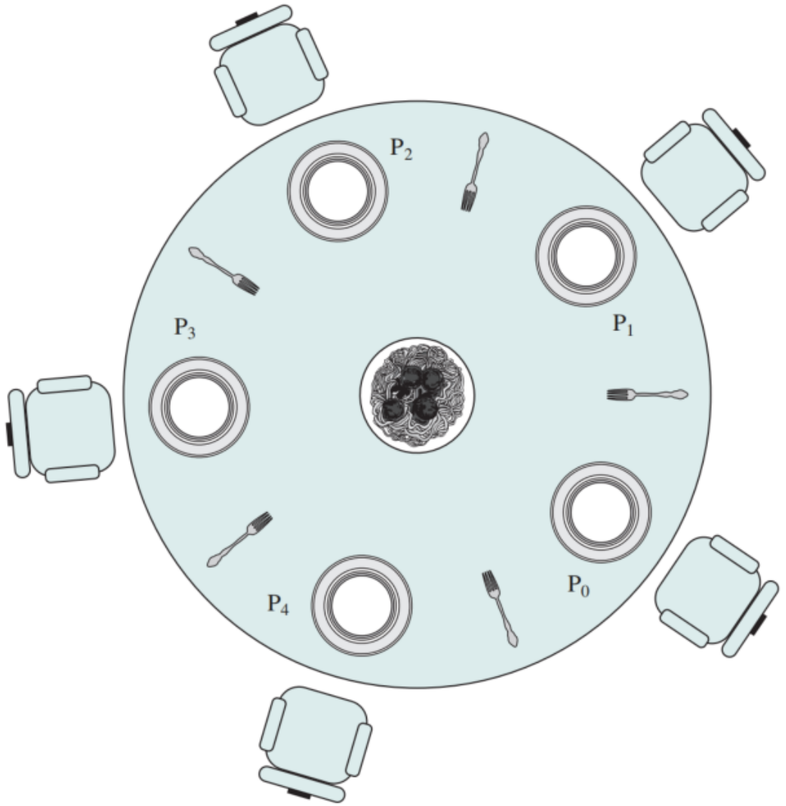
终止All死锁进程

- Abort all deadlocked processes （applied by most of OS）
- Back up each deadlocked process to some previously defined checkpoint and restart all processes 回滚到检查点，并重启所有死锁进程
- Successively abort deadlocked processes until deadlock no longer exists
- Successively preempt resources until deadlock no longer exists

Dining Philosophers Problem

- **Devise a algorithm that allow the philosophers to eat**
  - No two philosophers can use the same fork at the same time (mutual exclusion)
  - Avoid deadlock and starvation

using the two forks on either side of the plate, takes and eats some spaghetti



Solution Using Semaphores

- Each philosopher first picks up the fork on the left then the fork on the right.

```
/* program diningphilosophers */
semaphore fork [5] = {1};
int i;
void philosopher (int i)
{
     while (true) {
          think();
          wait (fork[i]);
          wait (fork [(i+1) mod 5]);
          eat();
          signal(fork [(i+1) mod 5]);
          signal(fork[i]);
     }
}
void main()
{
     parbegin (philosopher (0), philosopher (1),
          philosopher (2), philosopher (3),
          philosopher (4));
     }
```

A Second Solution . . .

- consider adding an attendant who only allows four philosophers at a time into the dining room.

增加服务员

```
/* program diningphilosophers */
semaphore fork[5] = {1};
semaphore room = {4};
int i;
void philosopher (int  i)
{
     while (true) {
          think();
          wait (room);
          wait (fork[i]);
          wait (fork [(i+1) mod 5]);
          eat();
          signal (fork [(i+1) mod 5]);
          signal (fork[i]);
          signal (room);
     }
}
void main()
{
     parbegin  (philosopher (0), philosopher (1),
          philosopher (2), philosopher (3),
          philosopher (4));
 }
```

Solution Using A Monitor

管程

```
monitor dining_controller;
cond ForkReady[5];    /* condition variable for synchronization */
boolean fork[5] = {true}; /* availability status of each fork */

void get_forks(int pid)    /* pid is the philosopher id number */
{
    int left = pid;
    int right = (++pid) % 5;
    /*grant the left fork*/
    if (!fork[left])
        cwait(ForkReady[left]); /* queue on condition variable */
    fork(left) = false;
    /*grant the right fork*/
    if (!fork[right])
        cwait(ForkReady[right]);/* queue on condition variable */
    fork[right] = false:
    }
void release_forks(int  pid)
{
    int left = pid;
    int right = (++pid) % 5;
    /*release the left fork*/
    if (empty(ForkReady[left]) /*no one is waiting for this fork */
        fork[left] = true;
    else                /* awaken a process waiting on this fork */
        csignal(ForkReady[left]);
    /*release the right fork*/
    if (empty(ForkReady[right])/*no one is waiting for this fork */
        fork[right] = true;
    else                /* awaken a process waiting on this fork */
        csignal(ForkReady[right]);
}
```

```
void philosopher[k=0 to 4]    /* the five philosopher clients */
{
    while  (true) {
      <think>;
     get_forks(k);    /* client requests two forks via monitor */
      <eat spaghetti>;
      release_forks(k); /* client releases forks via the monitor */
    }
}
```

Will deadlock happen using monitor?
How many philosopher can eat together?

Summary

Deadlock:

- the blocking of a set of processes that either compete for system resources or communicate with each other
- blockage is permanent unless OS takes action
- may involve reusable or consumable resources
- Consumable = destroyed when acquired by a process
- Reusable = not depleted/destroyed by use

Dealing with deadlock:

- prevention – guarantees that deadlock will not occur
- avoidance – analyzes each new resource request (Banker algorithm)
- detection – OS checks for deadlock and takes action (recovery)